Enterprise IT Asset Management System (Project "Tracer")

Architecture & Business Capabilities Blueprint

1. Executive Summary & System Vision

This document establishes the architectural foundation and business capability blueprint for **Tracer**, an enterprise-grade IT Asset Management (ITAM) system. While inspired by the comprehensive functional surface of Snipe-IT, Tracer is designed from the ground up to eliminate monolithic constraints. It introduces a highly scalable, audit-compliant, and event-driven architecture tailored for modern enterprise environments.
The system enforces strict asset lifecycle governance, financial accountability, and regulatory compliance. It achieves this while offering real-time user experiences powered by a modern, decoupled technology stack.

2. Global Architecture & Technology Stack

Tracer is built on a split-architecture pattern. It separates the presentation layer from the core business logic, persistence, and processing layers.

Code snippet

```
graph TD
  subgraph Client Layer [Presentation Layer: Angular 20+]
    UI[Angular SPA / Material] --> Signals[Angular Signals State Management]
    Signals --> HTTP[HttpClient / REST API Clients]
  end

  subgraph API Gateway / Edge
    Gateway[Reverse Proxy / API Gateway]
  end

  subgraph Application Layer [ASP.NET Core 9 Web API]
    HTTP --> Gateway
    Gateway --> Controllers[API Controllers / Minimal APIs]

    subgraph Clean Architecture Core
      Controllers --> Commands[MediatR Commands / Queries]
```

```
        subgraph CQRS Boundary
            Commands --> Handlers[Command / Query Handlers]
            Handlers --> Domain[Domain Entities & Aggregates]
            Handlers --> Infrastructure[Infrastructure Services]
        end
    end
end

subgraph Persistence Layer
    Infrastructure --> EF[EF Core 9]
    EF --> DB[(SQL Server)]
end

style Client Layer fill:#f9f,stroke:#333,stroke-width:2px
style Application Layer fill:#bbf,stroke:#333,stroke-width:2px
style Persistence Layer fill:#dfd,stroke:#333,stroke-width:2px
```

2.1 Backend Architecture: ASP.NET Core 9 Clean Architecture & CQRS

The backend follows **Clean Architecture** principles to isolate business logic from external frameworks, databases, and delivery mechanisms. **CQRS (Command Query Responsibility Segregation)** is implemented via **MediatR** to split read and write pathways, optimizing performance and maintainability.

- **Domain Layer:** Contains enterprise aggregates, entities, value objects, domain exceptions, and core domain rules. It has zero external dependencies.
- **Application Layer:** Defines use cases, MediatR command/query handlers, validation pipelines (FluentValidation), and repository interfaces.
- **Infrastructure Layer:** Implements data persistence via **Entity Framework Core 9**, integration services (directory sync, mailing), and security configurations.
- **API Layer:** ASP.NET Core 9 Minimal APIs / Controllers serving as thin entry points, translating HTTP requests into MediatR structures.

2.2 Frontend Architecture: Angular 20+ Reactive Architecture

The frontend leverages the performance enhancements of **Angular 20+**, moving away from legacy change detection in favor of a fully reactive paradigm.

- **State Management:** Driven entirely by **Angular Signals** for granular, high-performance DOM updates without global zone pollution.
- **UI Framework: Angular Material** configured with customized enterprise-grade design tokens for responsive, accessible dashboards.
- **Component Pattern:** Strict separation between Smart (Container) components handling data streams and Dumb (Presentation) components handling UI interactions.

2.3 Storage Layer: SQL Server

A relational model built on **SQL Server**, utilizing advanced indexing, row-level security (RLS) for multi-tenancy configurations, temporal tables for historical system auditing, and optimized execution paths for complex asset hierarchies.

3. Core Business Capability Map

The system is divided into modular domains, each mapping to specific Snipe-IT capabilities and expanded for enterprise operations.

Capability Module	Core Functional Scope	Enterprise Extensions
3.1 Identity & Access Management (IAM)	User, Role, and Group provisions; Snipe-IT permission matrix.	Native OIDC/OAuth2, Azure AD/Entra ID automated SCIM provisioning, RBAC down to field-level.
3.2 Asset Lifecycle Management	Hardware provisioning, check-in/check-out, status labels, asset tags.	Automated hardware deprecation models, multi-stage disposal workflows, bulk lifecycle transitions.
3.3 License & Subscription Governance	Software seat tracking, license keys, expiration alerts.	SaaS subscription cost optimization, over-allocation alerts, vendor true-up compliance reports.
3.4 Component & Accessory Control	RAM, HDD, peripherals tracking; bulk-consumable counts.	Automated low-stock procurement triggers, serialization of high-value components.
3.5 Digital Asset & Consumable Logistics	Toner, cables, digital certificates, software media tracking.	Expiration tracking for SSL/TLS certificates, FIFO inventory queues for physical consumables.
3.6 Financial Engineering & Enterprise Hierarchy	Deprecation schedules, company/location/department structuring.	Multi-currency support, tax jurisdiction depreciation, cost-center cross-charging.

3.7 Multi-Channel Notification Engine	Email check-out alerts.	Webhooks, Slack/Teams interactive cards, SMS backup notifications.
3.8 System Auditing & Compliance Matrix	Activity logging, physical asset audit schedules.	Cryptographically signed audit trails, automated barcode/RFID scanning reconciliation workflows.

4. Domain Deep-Dives: Workflows, Rules, and Specifications

4.1 Asset Lifecycle Management Workflow

Assets move through a definitive state machine. The system protects against illegal state transitions (e.g., checking out an asset marked as *Archived* or *Under Repair*).

Code snippet
stateDiagram-v2
[*] --> ReadyToDeploy : Inventory Ingest (Pending)
ReadyToDeploy --> Deployed : Check-Out (Assign to User/Location/Asset)
Deployed --> ReadyToDeploy : Check-In (Return to Inventory)
ReadyToDeploy --> UnderRepair : Damage / Maintenance Trigger
UnderRepair --> ReadyToDeploy : Repair Complete
ReadyToDeploy --> Archived : End of Life / Decommissioned
Archived --> [*]

Critical Business Rules:

- **Unique Asset Tagging:** Every hardware asset must possess a globally unique, non-modifiable Asset Tag.
- **Status Label Constraints:** Only assets mapped to a Status Label with the meta-type Deployable can be checked out. Assets marked as Pending, Archived, or Undeployable must block all deployment commands.
- **Circular Checkout Prevention:** An asset cannot be checked out to itself or to a child asset within a nested hardware tree structure.

4.2 Software License Allocation Engine

Manages software seats efficiently. When a license is checked out, the available seat pool decrements dynamically.

Code snippet
sequenceDiagram
autonumber
actor Admin as IT Administrator
participant API as License Controller
participant Handler as CheckOutLicenseCommandHandler
participant DB as SQL Server

Admin->>API: Post Check-Out Request (LicenseID, TargetUserID)
API->>Handler: Send Command via MediatR
activate Handler
Handler->>DB: Fetch License Aggregate & Current Seat Allocations
DB-->>Handler: License Data Returned

alt No Seats Available
Handler-->>API: Throw InsufficientSeatsException
API-->>Admin: HTTP 422 Unprocessable Entity (Error Details)
else Seats Available & Target Eligible
Handler->>Handler: Decrement Available Count & Assign Seat
Handler->>DB: Persist Allocation & Append History Log
DB-->>Handler: Commit Transaction
Handler-->>API: Return Success Payload
API-->>Admin: HTTP 200 OK
end
deactivate Handler

Critical Business Rules:

- **Seat Over-allocation Protection:** If Available_Seats <= 0, any further checkout attempts must be rejected by the validation pipeline before generating database locks.
- **Reassignment Rules:** Product keys marked as non-transferable (e.g., OEM licenses) cannot be checked back into inventory once deployed to a host hardware entity.

4.3 Identity, Role-Based Access Control (RBAC), and SCIM Synchronizer

Tracer implements a granular permission model mirroring Snipe-IT's matrix, elevated for enterprise federated identity structures.

Code snippet
graph LR

```
  IdP[Identity Provider: Azure AD / Okta] -->|SCIM 2.0 Protocol| SCIM[SCIM Endpoint / Auth Middleware]
  SCIM -->|Upsert User / Sync Groups| UserDB[(User & Role Store)]
  UserDB -->|Evaluate Claims| AuthZ[Authorization Pipeline]
  AuthZ -->|Access Granted / Denied| Execution[API Command / Query Execution]
```

Critical Business Rules:

- **SCIM Primacy:** When SCIM synchronization is enabled for a user directory, profile modifications (Name, Email, Department) are locked in the UI and can only be updated via the identity provider.
- **Self-Demotion Guardrail:** A user cannot modify or revoke their own administrative roles or permissions.

5. Frontend Signal Architecture Blueprint

The UI ensures immediate interface responsiveness by managing state transitions locally. The architecture dictates a strict separation between data fetching, state holding, and UI rendering, entirely bypassing legacy object-oriented state classes.

```
Code snippet
graph TD
  subgraph UI Components
    Smart[Smart Container Component]
    Dumb[Presentation Component]
  end

  subgraph State Management
    Store[Signal Store Service]
    Writable[Writable Signals: State]
    Computed[Computed Signals: Derived State]
  end

  subgraph Network Layer
    HTTP[Angular HTTP Client]
    API[Backend MediatR API]
  end

  Smart -->|Dispatches Action| Store
  Smart -->|Binds to| Computed
  Store -->|Mutates| Writable
  Writable -->|Updates| Computed
```

```
Store -->|Fetches Data| HTTP
HTTP <--> API
Smart -->|Passes Data via Inputs| Dumb
Dumb -->|Emits Events via Outputs| Smart
```

Critical Architecture Rules:

- **Immutability of State:** Components may only read from computed signals. All state mutations must occur strictly through designated update methods within the injectable store services.
- **Event-Driven Updates:** Network responses map directly to Signal updates, triggering highly localized DOM repaints without traversing the entire component tree.

6. Verification and Audit Matrix

To comply with global financial and IT governance regulations (such as SOC2, ISO 27001, and ITIL v4), Tracer includes an immutable verification and audit system.

- **Temporal Table Logging:** Every mutation within the database is historicalized automatically using SQL Server Temporal Tables, tracking the exact state changes alongside the executing application user identity context.
- **Physical Audit Cycle:** Administrators can create an Audit Schedule targeting physical sites. Technicians scan asset tags via the mobile interface, executing a specialized MediatR command (ReconcileAssetAuditCommand) that marks the physical asset as verified on that specific timestamp.

7. Documentation Readiness Checklist

This architectural design satisfies the foundational requirements for development alignment:

- **Architects:** Validates Clean Architecture boundaries, data isolation, and CQRS patterns.
- **Developers:** Provides clear implementation guidelines for MediatR command chains and reactive Angular Signals management without relying on concrete code implementations.
- **Testers:** Outlines core business logic, valid/invalid state transitions, and expected edge-case exception models.
- **Business Analysts:** Maps technical modules directly to the operational capabilities required for enterprise IT asset governance.